

R.Vishnu Vardhan - 2520090166

● TASK 1



```
root@Vishnu: ~
GNU nano 7.2 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    pid_t pid;
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        printf("\n--- Child Process ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        execlp(command, command, NULL);

        printf("Error: Invalid command!\n");
        exit(1);
    }

    else {
        printf("\n--- Parent Process ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }
}
```

[Read 48 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```
root@Vishnu:~# nano task1.c
root@Vishnu:~# gcc task1.c -o task1
root@Vishnu:~# ./task1
Enter a Linux command: ls

--- Parent Process ---

--- Child Process ---
Parent PID : 1101
Child PID : 1102
Child PID : 1102
Parent PID : 1101
OOSP a a.c.save b c d.c file1.txt fork.c task1.c
OSSP a.c a.c.save.1 b.c c.c d.c.save fork task1

Child process has finished execution.
root@Vishnu:~# |
```

● TASK 2

1. UNAME

```
root@Vishnu: ~
root@Vishnu:~# uname -a
Linux Vishnu 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64
GNU/Linux
root@Vishnu:~#
```

2. lScpu

```
root@Vishnu:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          48 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 4
On-line CPU(s) list:   0-3
Vendor ID:              AuthenticAMD
Model name:             AMD Ryzen 3 3250U with Radeon Graphics
CPU family:            23
Model:                 24
Thread(s) per core:    2
Core(s) per socket:    2
Socket(s):              1
Stepping:               1
BogoMIPS:               5190.10
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr ss
e sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology
tsc_rdtscp nonstop_tsc cpuid extd_apicid tsc_known_freq pni pclmulqdq sse4_1 sse4_2 movbe popcnt aes xsave
avx f16c rdrand hypervisor lahf_lm cmp_legacy svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext
perfctr_core ssbd ibpb vmcall fsgsbase bti1 avx2 smp bmi2 rdseed adx smap clflushopt sha_ni xsaveopt xsavec
xgetbv1 clzero xsave
erptr virt ssbd arat nrip_save tsc_scale vmcb_clean flushbyasid decodeassists pausefil
ter pfthreshold v_vmsave_vmload

Virtualization features:
Virtualization:        AMD-V
Hypervisor vendor:     Microsoft
Virtualization type:   full
Caches (sum of all):
L1d:                   64 KiB (2 instances)
L1i:                   128 KiB (2 instances)
L2:                    1 MiB (2 instances)
L3:                    4 MiB (1 instance)
NUMA:
NUMA node(s):          1
NUMA node0 CPU(s):    0-3
Vulnerabilities:
Gather data sampling:  Not affected
Ghostwrite:           Not affected
Indirect target selection: Not affected
Itlb multihit:        Not affected
L1tf:                 Not affected
Mds:                  Not affected
Meltdown:             Not affected
Mmio stale data:      Not affected
Old microcode:        Not affected
Reg file data sampling: Not affected
Retbleed:             Mitigation; untrained return thunk; SMT vulnerable
Spec rstack overflow: Vulnerable; Safe RET, no microcode
Spec store bypass:    Mitigation; Speculative Store Bypass disabled via pctl
Spectre v1:           Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:           Mitigation; Retpolines; IBPB conditional; STIBP disabled; RSB filling; PBRSE-eIBRS Not affe
```

3. Ps

```
root@Vishnu: ~  
root@Vishnu:~# ps -ef  
UID          PID     PPID  C  STIME TTY          TIME CMD  
root          1        0  2  04:59 ?        00:00:03 /sbin/init  
root          2        1  0  04:59 hvc0    00:00:00 /init  
root          6        2  0  04:59 hvc0    00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --  
root         64        1  0  04:59 ?        00:00:00 /usr/lib/systemd/systemd-journald  
root        118        1  0  04:59 ?        00:00:00 /usr/lib/systemd/systemd-udevd  
systemd+    123        1  0  04:59 ?        00:00:00 /usr/lib/systemd/systemd-resolved  
systemd+    124        1  0  04:59 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd  
root        172        1  0  04:59 ?        00:00:00 /usr/sbin/cron -f -P  
message+    173        1  0  04:59 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --syst  
root        193        1  0  04:59 ?        00:00:00 /usr/lib/systemd/systemd-logind  
root        202        1  0  04:59 ?        00:00:00 /usr/libexec/wsl-pro-service -vv  
syslog      218        1  0  04:59 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE  
root        232        1  0  04:59 tty1    00:00:00 /sbin/agetty -o -p -- \u --noclear - linux  
root        240        1  0  04:59 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-s  
root        361        2  0  04:59 pts/1   00:00:00 /bin/login -f  
root        434        1  1  04:59 ?        00:00:01 /usr/lib/systemd/systemd --user  
root        440       434  0  04:59 ?        00:00:00 (sd-pam)  
root        487       361  0  04:59 pts/1   00:00:00 -bash  
root        658        2  0  05:01 ?        00:00:00 /init  
root        659       658  0  05:01 ?        00:00:00 /init  
root        662       659  0  05:01 pts/0    00:00:00 -bash  
root        681       662  0  05:01 pts/0    00:00:00 ps -ef  
root@Vishnu:~#
```

4. Top

```
root@Vishnu: ~  
top - 05:02:18 up 3 min, 1 user, load average: 0.19, 0.33, 0.16  
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie  
Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st  
Mem Mem : 2074.6 total, 1803.2 free,  420.5 used,  636.0 buff/cache  
Mem Swap: 1024.0 total, 1024.0 free,  0.0 used,  2446.1 avail Mem  
  
  PID USER      PR  NI   VIRT    RES    SHR   S  %CPU  %MEM    TIME+  COMMAND  
    1 root        20   0  21852  13072  9732   S   0.0   0.4  0:03.84 systemd  
    2 root        20   0   3180   2204  2072   S   0.0   0.1  0:00:26 init-systemd(Ub  
    6 root        20   0   3196   2128  2008   S   0.0   0.1  0:00:01 init  
   64 root       19  -1  33972  12568  11508   S   0.0   0.4  0:00:73 systemd-journal  
  118 root        20   0  25464   6840   5132   S   0.0   0.2  0:00:47 systemd-udevd  
  123 systemd+    20   0  21344  13232  11104   S   0.0   0.4  0:00:36 systemd-resolve  
  124 systemd+    20   0   9186   7904   6948   S   0.0   0.3  0:00:21 systemd-timesyn  
  172 root        20   0   4244   2684   2432   S   0.0   0.1  0:00:02 cron  
  173 message+    20   0   9544   5492   4792   S   0.0   0.2  0:00:18 dbus-daemon  
  193 root        20   0  17980   8836   7796   S   0.0   0.3  0:00:19 systemd-logind  
  202 root        20   0 1156104  13048  10888   S   0.0   0.4  0:00:53 wsl-pro-service  
  218 syslog      20   0  222516  5972   4580   S   0.0   0.2  0:00:10 rsyslogd  
  232 root        20   0   3124   1972   1836   S   0.0   0.1  0:00:01 agetty  
  240 root        20   0 107032  23116  13664   S   0.0   0.8  0:00:47 unattended-upgr  
  361 root        20   0   6676   4524   3752   S   0.0   0.2  0:00:04 login  
  434 root        20   0  20316  11552  9432   S   0.0   0.4  0:01:42 systemd  
  440 root        20   0  21168   3508   1796   S   0.0   0.1  0:00:00 (sd-pam)  
  487 root        20   0   6080   5300   3676   S   0.0   0.2  0:00:11 bash  
  684 root        20   0   3184   1116    980   S   0.0   0.0  0:00:00 SessionLeader  
  688 root        20   0   3200   1256   1108   S   0.0   0.0  0:00:00 Relay(688)  
  688 root        20   0   6076   5380   3696   S   0.0   0.2  0:00:05 bash  
  707 root        20   0   9432   5748   3492   R   0.0   0.2  0:00:01 top
```

Relationship Between Hardware Resources and OS Services

1. CPU Abstraction

The CPU executes program instructions. Instead of allowing programs to access the processor directly, the operating system provides **process management**.

The OS:

- Creates processes
- Schedules CPU time
- Switches between multiple processes
- Shares CPU fairly among applications

Example:

- Chrome
- VS Code
- Terminal

All appear to run simultaneously because the OS rapidly switches the CPU between them.

2. Memory Abstraction

The operating system manages RAM through **virtual memory**.

Functions include:

- Allocating memory to processes
- Protecting one process from accessing another's memory
- Reclaiming unused memory
- Using swap space when RAM becomes full

Applications only see their allocated memory rather than the physical RAM.

3. Storage Abstraction

Storage devices such as SSDs and HDDs are managed through a **file system**.

The operating system:

- Creates files and folders
- Organizes data
- Reads and writes files
- Controls access permissions

Users interact with files instead of physical disk sectors.

4. I/O Device Abstraction

Input/output devices include:

- Keyboard
- Mouse
- Printer
- USB devices
- Monitor
- Network card

The operating system communicates with these devices through **device drivers**.

Applications simply request services like:

- Read keyboard input
- Display text
- Print a document

The OS handles all hardware communication internally.

Role of the Operating System

The operating system acts as an interface between the user and hardware by:

- Managing CPU scheduling
 - Managing memory allocation
 - Organizing files and storage
 - Controlling input/output devices
 - Providing security and protection
 - Managing multiple processes simultaneously
-

Observation

Using Linux commands, we observed that:

- `uname` provides kernel and operating system information.
- `lscpu` displays processor details.
- `ps` lists active processes managed by the OS.
- `top` shows real-time CPU and memory usage.

These commands demonstrate how the operating system manages hardware resources efficiently while allowing multiple applications to run smoothly.

Conclusion

The Linux operating system abstracts hardware resources by providing high-level services for CPU scheduling, memory management, storage organization, and device communication. This abstraction hides hardware complexity from users and applications, making the system easier to use, secure, and efficient. The commands `uname`, `lscpu`, `ps`, and `top` help observe how the operating system interacts with and manages the underlying hardware.